

```

// Project UID 1d9f47bfc76643019cfbf037641defe1
#include "Card.h"
#include "Pack.h"
#include "Player.h"

#include <string>
#include <vector>
#include <algorithm>
#include <cassert>
#include <fstream>
#include <stdio.h>
#include <string.h>
using namespace std;

class Game {
private:
    Pack p;
    Card upcard;
    std::vector <Player*> Play;
    bool is_dealer;
    int dealer_index; //initialize person in constructor
    int round;
    int hand;
    std::string trump;

public:
    int overall_team1_points;
    int overall_team2_points;
    Game(Pack& pack, int dealer_index, int round, int hand) :
        dealer_index(0), round(1), hand(0), overall_team1_points(0),
        overall_team2_points(0) {
        p = pack;
    }
    void Shuffle() {
        p.shuffle();
    }

    void Deal() {
        Play[(dealer_index + 1) % 4]->add_card(p.deal_one());
        Play[(dealer_index + 1) % 4]->add_card(p.deal_one());
        Play[(dealer_index + 1) % 4]->add_card(p.deal_one());
        Play[(dealer_index + 2) % 4]->add_card(p.deal_one());
        Play[(dealer_index + 2) % 4]->add_card(p.deal_one());
        Play[(dealer_index + 3) % 4]->add_card(p.deal_one());
        Play[(dealer_index + 3) % 4]->add_card(p.deal_one());
        Play[(dealer_index + 3) % 4]->add_card(p.deal_one());
        Play[(dealer_index + 3) % 4]->add_card(p.deal_one());
        Play[dealer_index]->add_card(p.deal_one());
        Play[dealer_index]->add_card(p.deal_one());

        Play[(dealer_index + 1) % 4]->add_card(p.deal_one());
        Play[(dealer_index + 1) % 4]->add_card(p.deal_one());
        Play[(dealer_index + 2) % 4]->add_card(p.deal_one());
        Play[(dealer_index + 2) % 4]->add_card(p.deal_one());
        Play[(dealer_index + 2) % 4]->add_card(p.deal_one());
        Play[(dealer_index + 3) % 4]->add_card(p.deal_one());
        Play[(dealer_index + 3) % 4]->add_card(p.deal_one());
        Play[dealer_index]->add_card(p.deal_one());
        Play[dealer_index]->add_card(p.deal_one());
        Play[dealer_index]->add_card(p.deal_one());
        Card upcard = p.deal_one();

        std::cout << "Hand " << hand << std::endl;
        std::cout << Play[dealer_index]->get_name() << " deals" <<

```

```

std::endl;
    std::cout << upcard << " turned up" << std::endl;
}

void updateHand() {
    ++hand;
}

void addPlayer(Player* pl) {
    Play.push_back(pl);
}

void updateDealer() {
    dealer_index = (dealer_index + 1) % 4;
}

void MakingTrump(const Card& upcard, bool is_dealer,
    int round, std::string& trump) {

    for (int i = 0; i < 4; ++i) {
        if (Play[(dealer_index + i) % 4]->make_trump(upcard, is_dealer,
round, trump)){
            trump = upcard.get_suit();
            std::cout << Play[(dealer_index + i) % 4]->get_name() <<
                " orders up " << trump << std::endl;
            Play[dealer_index]->add_and_discard(upcard);
            return;
        }
        else
            std::cout << Play[(dealer_index + i) % 4]->get_name() <<
                " passes" << std::endl;
    }

    for (int i = 0; i < 4; ++i) {
        if (Play[(dealer_index + i) % 4]->make_trump(upcard, is_dealer,
round, trump)){
            std::cout << Play[(dealer_index + i) % 4]->get_name() <<
                " orders up " << trump << std::endl;
            return;
        }
        else
            std::cout << Play[(dealer_index + i) % 4]->get_name() << "
passes" <<
                std::endl;
    }
}

int TrickTaking() {
    int leader;
    Card led_card;
    Card heighest_card;
    trump = led_card.get_suit();
    heighest_card = led_card;
    for (int i = 0; i < 4; ++i) {
        leader = i;
        led_card = Play[leader]->lead_card(trump);
        std::cout << led_card << " led by " << Play[leader]-
>get_name() <<
            std::endl;
        for (int p = 0; p < 3; ++p) {
            //Play[(leader + p) % 4]->play_card(led_card, trump);
            std::cout << Play[p]->play_card(heighest_card, trump) <<
                " played by " << Play[p]->get_name()

```

```

        << std::endl;

        if (Card_less(highest_card, Play[p]->play_card(led_card,
trump), trump)) {
            highest_card = Play[p]->play_card(led_card,
trump);

            leader = p;

            std::cout << Play[leader]->get_name() << " gets the
trick" << std::endl;

            std::cout << std::endl;
            return leader;
        }
    }
    return -1;
}

int score() {
    int Team1Score = 0;
    int Team2Score = 0;
    int team1NumTricks = 0;
    int team2NumTricks = 0;
    bool madeTrump = true;
    std::vector <Player*> Team1;
    std::vector <Player*> Team2;
    Team1.push_back(Play[0]);
    Team2.push_back(Play[1]);
    Team1.push_back(Play[2]);
    Team2.push_back(Play[3]);
    for (int i = 0; i < 4; ++i) {
        int winner = TrickTaking();
        if (winner == 0 || winner == 2) {
            ++team1NumTricks;
        }
        if (winner == 1 || winner == 3) {
            ++team2NumTricks;
        }
    }
    if (team1NumTricks > team2NumTricks) {
        if (madeTrump && (team1NumTricks == 3 || team1NumTricks == 4))
{
            Team1Score = 1;
            std::cout << Play[0]->get_name() << " and " <<
                Play[2]->get_name() << " win the hand" <<
std::endl;
            std::cout << Play[0]->get_name() << " and " << Play[2]-
>get_name() <<
                " have " << Team1Score << " points" << std::endl;
            std::cout << Play[1]->get_name() << " and " << Play[3]-
>get_name() <<
                " have " << Team2Score << " points" << std::endl;
            std::cout << std::endl;
            //return Team1Score;
        }
        if (team1NumTricks == 5) {
            Team1Score = 2;
            std::cout << Play[0]->get_name() << " and " << Play[2]-
>get_name() <<
                " win the hand" << std::endl;
            std::cout << "march!" << std::endl;
            std::cout << Play[0]->get_name() << " and " << Play[2]-
>get_name() <<

```

```

        " have " << Team1Score << " points" <<
std::endl;
std::cout << Play[1]->get_name() << " and " << Play[3]-
>get_name() <<
        " have " << Team2Score << " points" <<
std::endl;
        //return Team1Score;
    }

    if (!madeTrump && team1NumTricks <= 3) {
        Team1Score = 2;
        std::cout << Play[0]->get_name() << " and " << Play[2]-
>get_name() <<
            " win the hand" << std::endl;
        std::cout << "euchred!" << std::endl;
        std::cout << Play[0]->get_name() << " and " << Play[2]-
>get_name() <<
            " have " << Team1Score << " points" << std::endl;
        std::cout << Play[1]->get_name() << " and " << Play[3]-
>get_name() <<
            " have " << Team2Score << " points" << std::endl;
        std::cout << std::endl;
        //return Team1Score;
    }

}
if (team2NumTricks > team1NumTricks) {
    if (madeTrump && (team2NumTricks == 3 || team2NumTricks == 4))
{
        Team2Score = 1;
        std::cout << Play[1]->get_name() << " and " << Play[3]-
>get_name() <<
            " win the hand" << std::endl;
        std::cout << Play[1]->get_name() << " and " << Play[3]-
>get_name() <<
            " have " << Team2Score << " points" << std::endl;
        std::cout << Play[0]->get_name() << " and " << Play[2]-
>get_name() <<
            " have " << Team1Score << " points" << std::endl;
        std::cout << std::endl;
        //return Team2Score;
    }
    if (team2NumTricks == 5) {
        Team2Score = 2;
        std::cout << Play[1]->get_name() << " and " << Play[3]-
>get_name() <<
            " win the hand" << std::endl;
        std::cout << "march!" << std::endl;
        std::cout << Play[1]->get_name() << " and " << Play[3]-
>get_name() <<
            " have " << Team2Score << " points" << std::endl;
        std::cout << Play[0]->get_name() << " and " << Play[2]-
>get_name() <<
            " have " << Team1Score << " points" << std::endl;
        std::cout << std::endl;
        //return Team2Score;
    }
}

    if (!madeTrump) {
        if (team2NumTricks <= 3) {
            Team2Score = 2;
            std::cout << Play[1]->get_name() << " and " <<
Play[3]->get_name() <<
                " win the hand" << std::endl;

```

```

        std::cout << "euchred!" << std::endl;
        std::cout << Play[1]->get_name() << " and " <<
Play[3]->get_name() <<
        " have 2 points" << std::endl;
        std::cout << Play[0]->get_name() << " and " <<
Play[2]->get_name() <<
        " have " << Team1Score << " points" <<
std::endl;
        std::cout << std::endl;
    }
}
}
if (Team1Score > Team2Score) {
    overall_team1_points += Team1Score;
}
else
    overall_team2_points += Team2Score;
return -1;
}

void RunGame(Pack& pack) {
    MakingTrump(upcard, is_dealer, round, trump);
    TrickTaking();
    score();
}

};

int main(int argc, char** argv) {
    int overall_game_points = 0;
    int dealer_index = 0;
    int round = 1;
    int hand = 0;
    for (int i = 0; i < 12; ++i) {
        std::cout << argv[i] << " ";
    }
    std::cout << endl;
    if (argc != 12) {
        std::cout << "Usage: euchre.exe PACK_FILENAME [shuffle|noshuffle] "
<<
        "POINTS_TO_WIN NAME1 TYPE1 NAME2 TYPE2 NAME3 TYPE3 " << "NAME4
TYPE4" <<
        std::endl;
    }
    std::ifstream fin;
    fin.open(argv[1]);

    Pack P(fin);
    Game myGame = Game(P, dealer_index, round, hand);
    myGame.addPlayer(Player_factory(argv[4], argv[5]));
    myGame.addPlayer(Player_factory(argv[6], argv[7]));
    myGame.addPlayer(Player_factory(argv[8], argv[9]));
    myGame.addPlayer(Player_factory(argv[10], argv[11]));
    if (strcmp(argv[2], "shuffle") == 0) {
        myGame.Shuffle();
        myGame.Deal();
    }
    else if (strcmp(argv[2], "noshuffle") == 0) {
        myGame.Deal();
        P.reset();
    }
    //else

```

```

        //{
        //    std::cout << "Usage: euchre.exe PACK_FILENAME [shuffle|noshuffle] "
<<
        //    "POINTS_TO_WIN NAME1 TYPE1 NAME2 TYPE2 NAME3 TYPE3 " << "NAME4
TYPE4" << std::endl;
        //}
        if (1 <= overall_game_points && overall_game_points <= 100) {
            overall_game_points = atoi(argv[3]);

        }
        //else
        //{
        //    std::cout << "Usage: euchre.exe PACK_FILENAME [shuffle|noshuffle] "
<<
        //    "POINTS_TO_WIN NAME1 TYPE1 NAME2 TYPE2 NAME3 TYPE3 " << "NAME4
TYPE4" << std::endl;
        //}
        if (strcmp(argv[5], "Human") != 0 && strcmp(argv[5], "Simple") != 0) {
            std::cout << "Usage: euchre.exe PACK_FILENAME [shuffle|noshuffle] "
<<
            "POINTS_TO_WIN NAME1 TYPE1 NAME2 TYPE2 NAME3 TYPE3 " << "NAME4
TYPE4" <<
            std::endl;
        }
        else if ((strcmp(argv[5], "Human") == 0 || strcmp(argv[5], "Simple") ==
0)) {
            //std::cout << argv[4] << " " << argv[5] << " ";
        }
        if (strcmp(argv[7], "Human") != 0 && strcmp(argv[7], "Simple") != 0) {
            std::cout << "Usage: euchre.exe PACK_FILENAME [shuffle|noshuffle] "
<<
            "POINTS_TO_WIN NAME1 TYPE1 NAME2 TYPE2 NAME3 TYPE3 " << "NAME4
TYPE4" <<
            std::endl;
        }
        else if ((strcmp(argv[7], "Human") == 0 || strcmp(argv[7], "Simple") ==
0)) {
            //std::cout << argv[6] << " " << argv[7] << " ";
        }
        if (strcmp(argv[9], "Human") != 0 && strcmp(argv[9], "Simple") != 0) {
            std::cout << "Usage: euchre.exe PACK_FILENAME [shuffle|noshuffle] "
<<
            "POINTS_TO_WIN NAME1 TYPE1 NAME2 TYPE2 NAME3 TYPE3 " << "NAME4
TYPE4" <<
            std::endl;
        }
        else if ((strcmp(argv[9], "Human") == 0 || strcmp(argv[9], "Simple") ==
0)) {
            //std::cout << argv[8] << " " << argv[9] << " ";
        }
        if (strcmp(argv[11], "Human") != 0 && strcmp(argv[11], "Simple") != 0) {
            std::cout << "Usage: euchre.exe PACK_FILENAME [shuffle|noshuffle] "
<<
            "POINTS_TO_WIN NAME1 TYPE1 NAME2 TYPE2 NAME3 TYPE3 " << "NAME4
TYPE4" <<
            std::endl;
        }
        else if ((strcmp(argv[11], "Human") == 0 || strcmp(argv[11], "Simple") ==
0)) {
            //std::cout << argv[10] << " " << argv[11] << std::endl;
        }
        if (myGame.overall_team1_points < atoi(argv[3]) &&
            myGame.overall_team2_points < atoi(argv[3])) {
            myGame.updateHand();
        }
    }
}

```

```
        myGame.updateDealer();
        myGame.RunGame(P);
    }
    if (!fin.is_open()) {
        std::cout << "Error opening " << argv[1] << std::endl;
    }
    return 0;
}
```